CYBERSECURITY PORTFOLIO SUMMARY

Nexzen E-Commerce Website Security Hardening

Full-stack security improvements delivered on a Next.js 16, Prisma, Supabase, and Razorpay commerce platform for electronics and IoT hardware sales.

ROLE	PLATFORM	VERIFICATION
Cybersecurity Engineer	**Next.js 16 / Prisma / Supabase**	**Production build passed**

Project Overview

Nexzen is a modern electronics storefront with catalog browsing, customer authentication, cart and checkout flows, Razorpay payment support, support tickets, wishlist, reviews, inventory controls, and an internal admin console for products, orders, banners, highlights, collections, brands, categories, and coupons.

Security Work Delivered

- Added global HTTP security headers including CSP, HSTS, X-Frame-Options, Referrer-Policy, Permissions-Policy, and nosniff.
- Disabled the `X-Powered-By` header and hardened proxy behavior for hidden admin routes.
- Implemented real admin login throttling with a hard stop at 15 password attempts inside a 10-minute window.
- Added CSRF protection for cookie-based admin actions using a dedicated CSRF cookie and required request header.
- Centralized admin request validation with IP allowlisting, session checks, origin checks, and CSRF enforcement.
- Hardened admin CRUD routes across products, orders, coupons, banners, categories, collections, highlights, and brand management.
- Introduced admin account recovery with OTP login and password reset options after 10 failed password attempts.

Data and Payment Protections

- Moved checkout pricing enforcement server-side so order totals are recalculated from database prices instead of trusting browser-submitted values.
- Validated coupon reuse against prior customer orders before order creation.
- Restricted Razorpay order creation to authenticated owners of the order and stopped trusting client-submitted amounts.
- Switched payment signature checks to `timingSafeEqual`-based verification.
- Restricted order lookup during payment to the authenticated owner instead of exposing order details publicly.
- Validated cart sync payloads and filtered invalid product IDs before database writes.

Input and Upload Hardening

- Added shared normalization helpers for text, email, decimals, integers, phone numbers, and Indian pincodes.
- Validated shipping, profile, and support-ticket inputs before persistence.
- Removed trust in client-supplied `userId` during support ticket submission and resolved identity from authenticated context instead.
- Validated uploaded images by MIME type and file size and restricted them to PNG, JPG, and WebP.

Authentication and Session Controls

- Preserved Supabase token verification for customer-protected APIs.
- Retained hashed admin passwords with scrypt and hashed session tokens in storage.
- Extended admin login to issue both secure session cookies and CSRF cookies together.
- Ensured admin logout clears both the admin session cookie and CSRF cookie.

- Constrained gallery URLs and asset links to HTTPS or local relative paths where applicable.
- Added persistent database-backed OTP challenges and single-use password reset tokens for admin recovery workflows.
- Kept hidden admin path rewriting and 404 responses for direct `/admin` access.
- Delivered email-based admin OTP and reset-link workflows using SMTP with expiry and one-time-use enforcement.

Build verification completed successfully after changes.

Technical Stack

Next.js 16 App Router, React 19, Tailwind CSS v4, Prisma ORM, PostgreSQL, Supabase Authentication, Razorpay, Nodemailer SMTP, and Vercel deployment.

Portfolio Summary

I secured a production-style e-commerce platform by hardening admin access, session handling, CSRF defenses, server-side checkout validation, payment integrity checks, HTTP response headers, file upload restrictions, and sensitive route authorization. This project demonstrates practical web security implementation across authentication, authorization, payment security, secure input handling, and defense-in-depth controls for a modern JavaScript commerce stack.

Prepared for portfolio use from the local project source after code-level review and implementation.